Report on

“Tutorials”

For

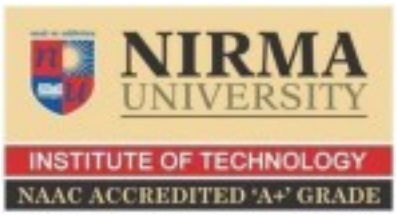
Software Project Management - 4CS504ME25 B.

Tech. Semester IV

Prepared By :

Gandhi Urva Yogeshkumar (23BCE078)

Gajnotar Rakshit (23BCE077)



Institute of Technology,
Nirma University
Ahmedabad - 382481

August 2026

Aim

To build an Activity on Node (AON) network diagram, calculate schedule parameters using the Critical Path Method (CPM) and prepare a 16-week Gantt chart with project milestones for the Hospital Management System (HMS).

Objective

- Break down the 9 deliverables and 23 activities from T4 into an activity sequence with duration estimates.
- Set out realistic scheduling assumptions based on our two-student team capacity.
- Calculate Earliest Start (ES), Earliest Finish (EF), Latest Start (LS), Latest Finish (LF) and Total Slack using CPM forward and backward passes.
- Find the critical path through the network and check slack across non-critical tasks.
- Draw an Activity on Node (AON) network diagram showing all module dependencies.
- Construct a 16-week Gantt chart mapped over eight two-week sprints.
- Define milestone verification checkpoints for tracking progress during the semester.

1. Introduction

Project scheduling puts the work planned earlier onto a concrete calendar timeline with clear start points, durations, dependencies and milestones. Without a schedule, we cannot see if the project is running on time or spot which tasks might delay the final submission.

This report follows the Project Management Plan from T3 and the Work Breakdown Structure from T4 for the Hospital Management System (HMS). The 16-week project runs across eight two-week sprints. We use two main scheduling tools:

- Activity on Node (AON) Network Diagram: Shows each task as a box and dependencies as arrows, so we can work out the critical path, early dates, late dates and slack.
- Gantt Chart: Places the same activities on a calendar grid to show task durations, parallel work between team members and delivery milestones.

Our team divides work between two members: Urva writes the backend Node.js and Express APIs, database queries and authentication. Rakshit develops the React frontend screens, forms and client state. Testing and integration are handled together.

2. Scheduling Assumptions

- Time Unit: Time is measured in calendar weeks, totaling 16 weeks (eight 2-week sprints) for the semester schedule.
- Team Capacity: Durations are realistic estimates for two students working alongside regular college classes.
- Dependency Type: All dependencies follow Finish to Start rules; an activity only starts when all its predecessors are done.
- Parallel Work: In each module sprint, backend API coding and frontend UI work happen in parallel during the first week.
- Module Verification: Testing takes place in the second week of each sprint once backend APIs and frontend screens are connected.

- Tool Availibility: Development laptops, local databases (MongoDB and PostgreSQL) and Git repositories are available without any setup delay.

3. Activity List and Dependencies

The Hospital Management System is broken into 23 activities from the deliverables in T4. Each module has a backend part and a frontend part that run concurrently in the first week, followed by testing in the second week.

ID	Activity Description	Duration (wk)	Predecessors	Owner
A1	Requirement gathering and sprint planning	1.0	None	Both
A2	System design and architecture (ER diagram, API structure, auth)	1.0	A1	Both
A3	Patient module, backend (CRUD APIs)	1.0	A2	Urva
A4	Patient module, frontend (registration and records)	1.0	A2	Rakshit
A5	Patient module, testing and verification	1.0	A3, A4	Both
A6	Doctor and department module, backend	1.0	A5	Urva
A7	Doctor and department module, frontend	1.0	A5	Rakshit
A8	Doctor and department module, testing and verification	1.0	A6, A7	Both
A9	Appointment booking module, backend	1.0	A8	Urva
A10	Appointment booking module, frontend	1.0	A8	Rakshit
A11	Appointment booking module, testing and verification	1.0	A9, A10	Both
A12	Pharmacy stock module, backend	1.0	A11	Urva
A13	Pharmacy stock module, frontend	1.0	A11	Rakshit
A14	Pharmacy stock module, testing and verification	1.0	A12, A13	Both
A15	Billing and invoice module, backend	1.0	A14	Urva
A16	Billing and invoice module, frontend	1.0	A14	Rakshit
A17	Billing and invoice module, testing and verification	1.0	A15, A16	Both
A18	Admin dashboard module, backend	1.0	A17	Urva
A19	Admin dashboard module, frontend	1.0	A17	Rakshit
A20	Admin dashboard module, testing and verification	1.0	A18, A19	Both
A21	System integration (combine all six modules)	1.0	A20	Both
A22	User acceptance testing (UAT)	0.5	A21	Both
A23	Deployment and final documentation	0.5	A22	Both

4. Schedule Calculation (Critical Path Method)

The Critical Path Method (CPM) works out the schedule through two passes over the network:

- Forward Pass: Finds the Earliest Start (ES) and Earliest Finish (EF) of each activity from the beginning of the project.

• ES = 0 for the first activity.

• ES = largest EF of all immediate predecessors.

• EF = ES + Duration.

- Backward Pass: Finds the Latest Finish (LF) and Latest Start (LS), meaning how late a task can finish without pushing back the final delivery date.

• LF = total project duration for the last activity.

• LF = smallest LS of all immediate successors.

• LS = LF - Duration.

- Slack (Float): The spare time of an activity:

• Slack = LS - ES = LF - EF. When slack is 0, the activity is on the critical path.

Worked Calculaton Example

Take activity A5 (Patient module testing): Its predecessors are A3 (backend) and A4 (frontend). Both A3 and A4 finish at week 3.0. The forward pass gives ES(A5) = max(3.0, 3.0) = 3.0 weeks and EF(A5) = 3.0 + 1.0 = 4.0 weeks. Working backward from successors A6 and A7, which both have LS = 4.0 weeks, LF(A5) = min(4.0, 4.0) = 4.0 weeks. This gives LS(A5) = 4.0 - 1.0 = 3.0 weeks and Slack = 3.0 - 3.0 = 0.0 weeks. Activity A5 is critical.

Take activity A21 (System integration): Its predecessor is A20 (Admin dashboard testing) which finishes at week 14.0. So ES(A21) = 14.0 weeks and EF(A21) = 14.0 + 1.0 = 15.0 weeks. The successor A22 (UAT) has LS = 15.0 weeks, so LF(A21) = 15.0 weeks, LS(A21) = 15.0 - 1.0 = 14.0 weeks and Slack = 0.0 weeks.

The full forward and backward pass results for all 23 activities are listed in the table below (all values in weeks):

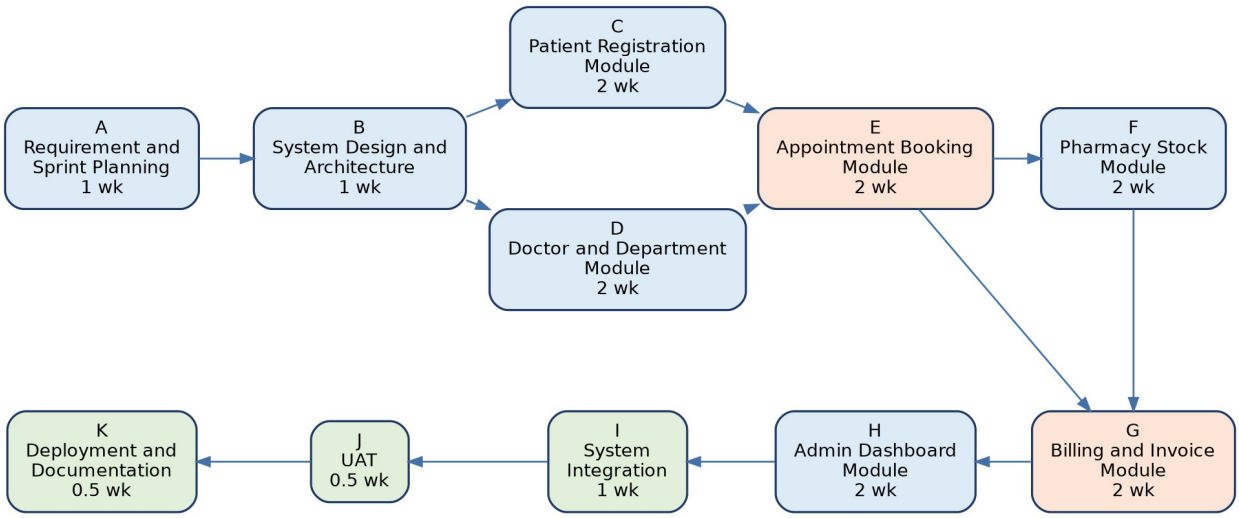
ID	Activity Description	Dur	Pred	ES	EF	LS	LF	Slack	Critical?
A1	Requirement gathering and sprint planning	1.0	None	0.0	1.0	0.0	1.0	0.0	**Yes**
A2	System design and architecture	1.0	A1	1.0	2.0	1.0	2.0	0.0	**Yes**
A3	Patient module, backend	1.0	A2	2.0	3.0	2.0	3.0	0.0	**Yes**
A4	Patient module, frontend	1.0	A2	2.0	3.0	2.0	3.0	0.0	**Yes**
A5	Patient module, testing	1.0	A3, A4	3.0	4.0	3.0	4.0	0.0	**Yes**
A6	Doctor and dept module, backend	1.0	A5	4.0	5.0	4.0	5.0	0.0	**Yes**
A7	Doctor and dept module, frontend	1.0	A5	4.0	5.0	4.0	5.0	0.0	**Yes**
A8	Doctor and dept module, testing	1.0	A6, A7	5.0	6.0	5.0	6.0	0.0	**Yes**
A9	Appointment module, backend	1.0	A8	6.0	7.0	6.0	7.0	0.0	**Yes**
A10	Appointment module, frontend	1.0	A8	6.0	7.0	6.0	7.0	0.0	**Yes**
A11	Appointment module, testing	1.0	A9, A10	7.0	8.0	7.0	8.0	0.0	**Yes**
A12	Pharmacy module, backend	1.0	A11	8.0	9.0	8.0	9.0	0.0	**Yes**

ID	Activity Description	Dur	Pred	ES	EF	LS	LF	Slack	Critical?
A13	Pharmacy module, frontend	1.0	A11	8.0	9.0	8.0	9.0	0.0	**Yes**
A14	Pharmacy module, testing	1.0	A12, A13	9.0	10.0	9.0	10.0	0.0	**Yes**
A15	Billing module, backend	1.0	A14	10.0	11.0	10.0	11.0	0.0	**Yes**
A16	Billing module, frontend	1.0	A14	10.0	11.0	10.0	11.0	0.0	**Yes**
A17	Billing module, testing	1.0	A15, A16	11.0	12.0	11.0	12.0	0.0	**Yes**
A18	Admin dashboard, backend	1.0	A17	12.0	13.0	12.0	13.0	0.0	**Yes**
A19	Admin dashboard, frontend	1.0	A17	12.0	13.0	12.0	13.0	0.0	**Yes**
A20	Admin dashboard, testing	1.0	A18, A19	13.0	14.0	13.0	14.0	0.0	**Yes**
A21	System integration (all 6 modules)	1.0	A20	14.0	15.0	14.0	15.0	0.0	**Yes**
A22	User acceptance testing (UAT)	0.5	A21	15.0	15.5	15.0	15.5	0.0	**Yes**
A23	Deployment and final documentation	0.5	A22	15.5	16.0	15.5	16.0	0.0	**Yes**

The last activity (A23) finishes at week 16.0, which sets the total project duration at 16 weeks.

5. Activity Network Diagram

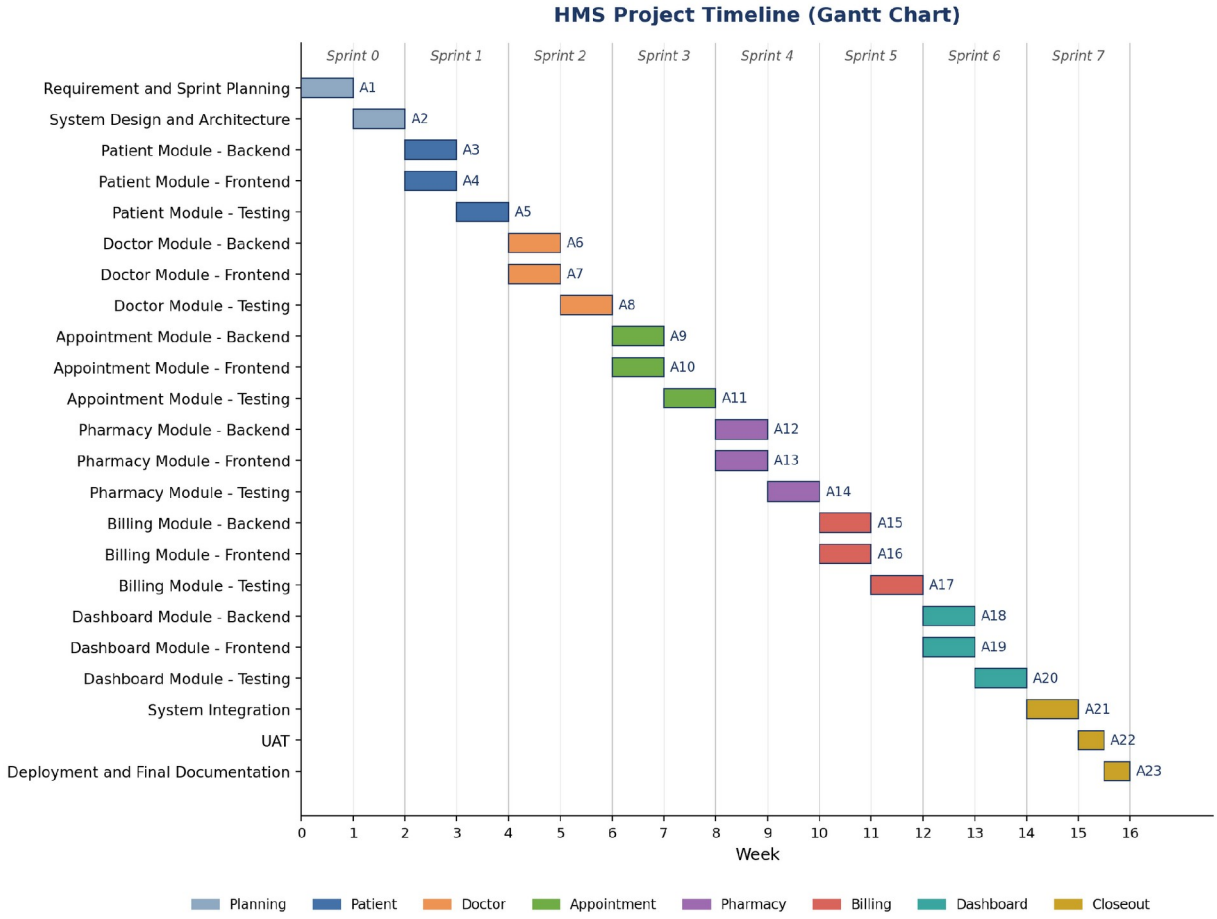
The diagram below shows the project at the deliverable level, matching the nine main deliverables from T4. Each box represents a deliverable with duration, early dates, late dates and float values. Arrows show the dependencies between deliverables.



Flow of work: Planning (A) and System Design (B) set up the core structure. Work then splits into Patient Registration (C) and Doctor and Department (D). Appointment Booking (E) needs both C and D finished first because bookings require valid patient IDs and doctor schedules. Pharmacy Stock (F) runs next and Billing (G) joins data from Appointment and Pharmacy to produce patient bills. Admin Dashboard (H) collects summaries across all modules. The project finishes with System Integration (I), User Acceptance Testing (J) and Deployment and Documentation (K).

6. Gantt Chart

The Gantt chart puts the activity schedule onto a calendar across the 16-week semester, divided into eight two-week sprints. During the first week of each module sprint, backend and frontend bars run in parallel. The second week is used for testing and review demos before the next sprint starts.



7. Critical Path Analysis

The critical path is the longest sequence of dependent activities required to complete the project. In the HMS schedule, the critical path runs through all activities:

A1 -> A2 -> A3/A4 -> A5 -> A6/A7 -> A8 -> A9/A10 -> A11 -> A12/A13 -> A14 -> A15/A16 -> A17 -> A18/A19 -> A20 -> A21 -> A22 -> A23 = 16.0 Weeks.

- Zero Slack: Every activity on the critical chain has Slack = 0.0 weeks. A delay in any backend, frontend or testing task pushes back the final completion date.
- Parallel Branches: Backend and frontend tasks in each sprint take 1.0 week each. Because both members work in parallel, neither track has spare float; both must finish by the end of week 1 to allow testing in week 2.
- Testing Checkpoints: Module testing tasks (A5, A8, A11, A14, A17, A20) act as quality checks where code reviews and bug fixes happen before moving to the next module.
- Work Distribution: Urva works on API routes and database models while Rakshit builds the React screens and forms. Working concurrently during development weeks lets the team complete 2 person-weeks of work per calendar week.

8. Project Milestones

Project milestones mark major checkpoints to verify progress against the baseline plan. Each milestone matches the end of a sprint review demo:

Milestone	Target Week / Sprint	Deliverable Checkpoint
M1: Inception and Design Frozen	End of Week 2 (Sprint 0)	SRS, ER diagrams, API routes and auth setup approved.
M2: Patient and Doctor Operational	End of Week 6 (Sprint 2)	Patient records and doctor roster working with database.
M3: Appointment Scheduling Live	End of Week 8 (Sprint 3)	Queue management, doctor booking and tokens verified.
M4: Pharmacy and Billing Integrated	End of Week 12 (Sprint 5)	Drug inventory stock linked to billing invoice creation.
M5: Dashboard and Modules Coded	End of Week 14 (Sprint 6)	Admin KPI summaries working across all six modules.
M6: Integration and UAT Completed	Mid Week 16 (Sprint 7)	End to end workflows tested with sample patient data.
M7: Final Release and Submission	End of Week 16 (Sprint 7)	System deployed locally, documentation finished and submitted.

9. Conclusion

The CPM calculaton shows that the Hospital Management System needs 16.0 weeks of elapsed time. Because backend and frontend tasks run in parallel across eight sprints, activities sit on the critical path with zero slack. Regular sprint demos and weekly testing help catch issues early so the schedule stays on track.